Resumen de Arquitectura MIPS

Eric Guacache

Nombres de registros, usos y convenio de llamadas

NOMBRE	NÚMERO	USO	¿SE CONSERVA?
`$zero`	0	Valor constante 0	N/A
`$at`	1	Ensamblador temporal	No
`$v0-$v1`	2-3	Valores de resultado de funciones y evaluación de expresiones	No
`$a0-$a3`	4-7	Argumentos	No
`$t0-$t7`	8-15	Temporales	No
`$s0-$s7`	16-23	Temporales almacenados	Sí
`$t8-$t9`	24-25	Temporales	No
`$k0-$k1`	26-27	Reservados para el núcleo del Sistema Operativo	No
`$gp`	28	Puntero global	Sí
`$sp`	29	Puntero de pila	Sí
`$fp`	30	Puntero de marco	Sí
`$ra`	31	Dirección de retorno	Sí

Formatos básicos de instrucción

Tipo	Campos					
R	cod oper (31-26)	rs (25-21)	rt (20-16)	rd (15-11)	desplaz (10-6)	func (5-0)
I	cod oper (31-26)	rs (25-21)	rt (20-16)	inmediato (15-0)		
J	cod oper (31-26)	dirección (25-0)				

Funciones de lenguaje ensamblador MIPS

Cuadro 1: Instrucciones MIPS.

Categoría	**Instrucción**	**Ejemplo**	**Significado**	**Comentarios**
Aritmética	add	`add $s1, $s2, $s3`	`$s1 = $s2 + $s3`	Tres operandos; datos en registros
	subtract	`sub $s1, $s2, $s3`	`$s1 = $s2 - $s3`	Tres operandos; datos en registros
	add immed.	`addi $s1, $s2, 100`	`$s1 = $s2 + 100`	Usado para sumar constantes
Transferencia de dato	load word	`lw $s1, 100($s2)`	`$s1 = Mem[$s2+100]`	Palabra de memoria a registro
	store word	`sw $s1, 100($s2)`	`Mem[$s2+100] = $s1`	Palabra de registro a memoria
	load half	`lh $s1, 100($s2)`	`$s1 = Mem[$s2+100]`	Media palabra de memoria a registro
	store half	`sh $s1, 100($s2)`	`Mem[$s2+100] = $s1`	Media palabra de registro a memoria
	load byte	`lb $s1, 100($s2)`	`$s1 = Mem[$s2+100]`	Byte de memoria a registro
	store byte	`sb $s1, 100($s2)`	`Mem[$s2+100] = $s1`	Byte de registro a memoria
	load upper immed.	`lui $s1, 100`	`$s1 = 100` $\cdot 2^{16}$	Cargar constante en los 16 bits de mayor peso
Lógica	and	`and $s1, $s2, $s3`	`$s1 = $s2 & $s3`	Tres registros operandos; AND bit-a-bit
	or	`or $s1, $s2, $s3`	`$s1 = $s2 | $s3`	Tres registros operandos; OR bit-a-bit
	nor	`nor $s1, $s2, $s3`	`$s1 =` $\sim$`($s2 | $s3)`	Tres registros operandos; NOR bit-a-bit
	and immed.	`andi $s1, $s2, 100`	`$s1 = $s2 & 100`	AND bit-a-bit registro con constante

Cuadro 1 – Continuación de la página anterior

Categoría	Instrucción	Ejemplo	Significado	Comentarios
	or immed.	`ori $s1, $s2, 100`	`$s1 = $s2 | 100`	OR bit-a-bit registro con constante
	shift left	`sll $s1, $s2, 10`	`$s1 = $s2` $\ll$ `10`	Desplazamiento a la izquierda por constante
	shift right	`srl $s1, $s2, 10`	`$s1 = $s2` $\gg$ `10`	Desplazamiento a la derecha por constante
Salto condicional	branch on eq	`beq $s1, $s2, L`	if (`$s1 == $s2`) go L	Comprueba igualdad y bifurca relativo al PC
	branch not eq	`bne $s1, $s2, L`	if (`$s1 != $s2`) go L	Comprueba si no igual y bifurca relativo al PC
	set on less	`slt $s1, $s2, $s3`	if (`$s2 <$s3`) `$s1=1` else `$s1=0`	Compara si es menor que; usado con beq, bne
	set on less imm	`slti $s1, $s2, 100`	if (`$s2 <100`) `$s1=1` else `$s1=0`	Compara si es menor que una constante
Salto incond.	jump	`j 2500`	go to 10000	Salto a la dirección destino
	jump reg	`jr $ra`	go to `$ra`	Para retorno de procedimiento
	jump and link	`jal 2500`	`$ra = PC+4`; go 10000	Para llamada a procedimiento